



THE UNIVERSITY OF
MELBOURNE

SWEN90010 - High Integrity Systems Engineering

Formal Specification

Toby Murray



toby.murray@unimelb.edu.au



DMD 8.17 (Level 8, Doug McDonnell Bldg)



<http://people.eng.unimelb.edu.au/tobym>



@tobycmurray



TEMPORAL LOGIC (TALKING ABOUT TIME)

Next State

if **x** is a variable, then **x'** refers to x's value
in the **next state**

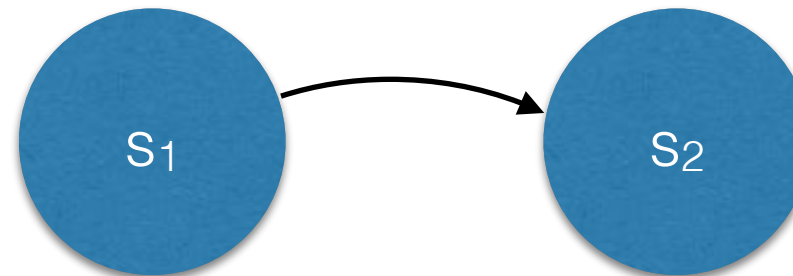
To specify that x is incremented, we would write

$$x' = x + 1$$

Predicates that talk about **x** are also allowed to talk
about x's value in **future states**, e.g. **x'**, **x''** etc.
and how it relates to the current value of x

These predicates define **state transitions**

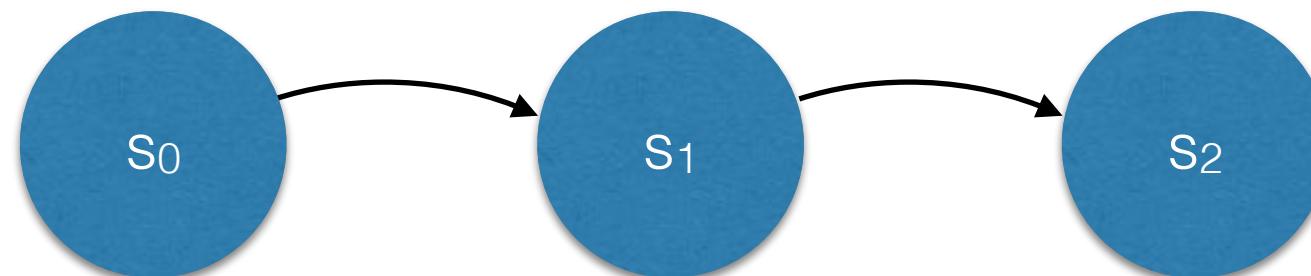
Transitions and Traces



$$x' = x + 1$$

If in s_1 , $x = 5$, then in s_2 , $x = 6$

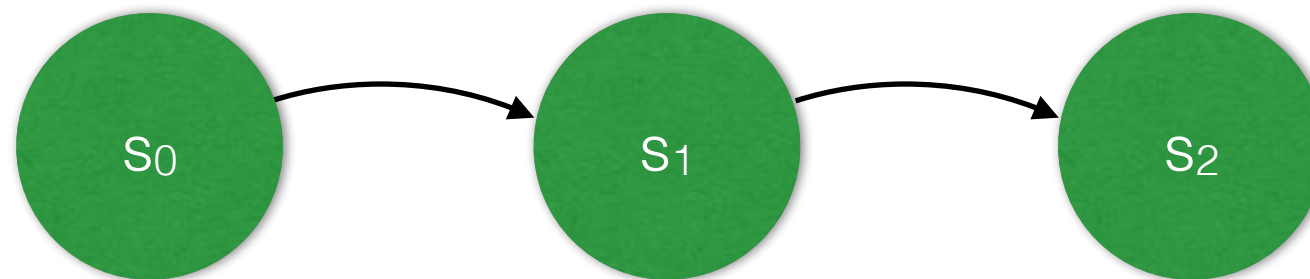
A sequence of transition is called a **trace**



Temporal Operators

We can also write predicates that talk about **future** and even **past** states using **temporal operators**

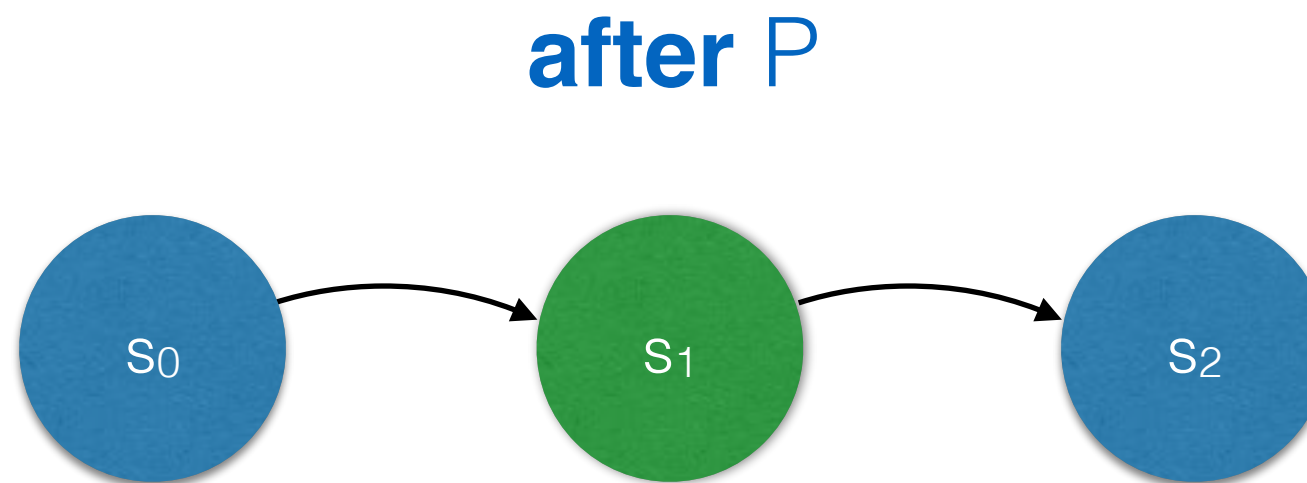
always P



assuming the current state is s_0

Temporal Operators

We can also write predicates that talk about **future** and even **past** states using **temporal operators**

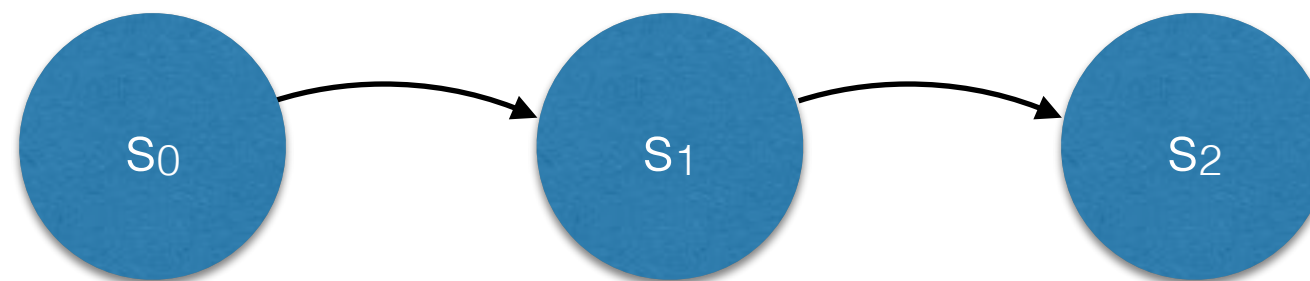


assuming the current state is s_0

Temporal Operators

We can also write predicates that talk about **future** and even **past** states using **temporal operators**

eventually P



P holds in at least one of these states
assuming the current state is s_0

Temporal Operators

We can also write predicates that talk about **future** and even **past** states using **temporal operators**

$P; Q$

equivalent to P **and** (**after** Q)

Temporal Operators



We can also write predicates that talk about **future** and even **past** states using **temporal operators**

We will meet others in this subject.
See course notes for more information



**SPECIFICATIONS SAY “WHAT”
DESIGNS SAY “HOW”**

Specifying Software

Specify **modules**

Formally describe (an abstraction of) the module's **state**

Formally describe the **behaviour** of the module's operations
(e.g. procedures, functions etc.).

Behaviour specified in terms of **state (transition) relations**.

	State:	Operation:
Trivial example : a counter	natural number $n : \mathbb{N}$	increment $n' = n + 1$

Modelling Data Types

Data types modelled as sets, relations, functions etc.

Example: a sequence (e.g. list, array, etc.) of T s

A **function**: $\mathbb{Z}^+ \rightarrow T$ i.e. a set of pairs: $(index, t)$

Sequence Operations: for a sequence seq

$seq(i)$

$seq[i]$

$length(seq)$

$\#seq$

$append(seq, el)$

$seq ++ ((\#seq + 1) \rightarrow el)$

Sequences as Relations

For a sequence $seq : \mathbb{Z}^+ \rightarrow T$

$domain(seq)$ valid list indices, i.e. $\{1..\#seq\}$

$range(seq)$ set of elements in the list

$seq :=> t$ $\{(index, e) : seq \mid e = t\}$

$\#(seq :=> t)$ number of occurrences of t

index	element
1	e_1
2	e_2
3	e_3
...	...
$\#seq$	e_n

Example: Searching

Searching a sorted list (e.g. binary search)

-- returns -1 if target doesn't exist in list

-- assumes the list is sorted

```
procedure Search(list : in ElementList; target : in Element;  
                index : out Integer);
```

Suppose we call `Search(list, t, index)`

What **relationship** should hold now between
`list`, `t` and `index`?

Should this
hold for **all** lists?

`list[index] = t`
or
`index = -1 and t not in range(list)`

Pre and Postconditions

Precondition: Specifies the **assumptions** made by a procedure/function

Postcondition: Specifies the **behaviour**, assuming that the precondition holds

Precondition for binary search: list is sorted

$\text{sorted}(\text{list}) \Leftrightarrow \mathbf{all} \ i : \mathbb{Z} \mid i < \#list \ \mathbf{implies} \ list[i] \leq list[i+1]$

Example: Searching again

Searching a sorted list (e.g. binary search)

-- returns -1 if target doesn't exist in list

-- assumes the list is sorted

```
procedure Search(list : in ElementList; target : in Element;  
                index : out Integer);
```

Suppose we call `Search(list, t, index)`

What **relationship** should hold now between

`list`, `t` and `index`?

postcondition

`sorted(list)` **and**

precondition

`list[index] = t`

or

`index = -1 and t not in range(list)`



Model the behaviour of each operation as a **relation**

The relation captures the relationship between the operation's inputs and outputs

In general:

relates the **states before and after the operation**

Says how they are allowed to differ (**postcondition**)
under assumptions about the inputs (**precondition**)

Says **what** the operation does, not **how** it does it.

Advantages

Small

Unambiguous

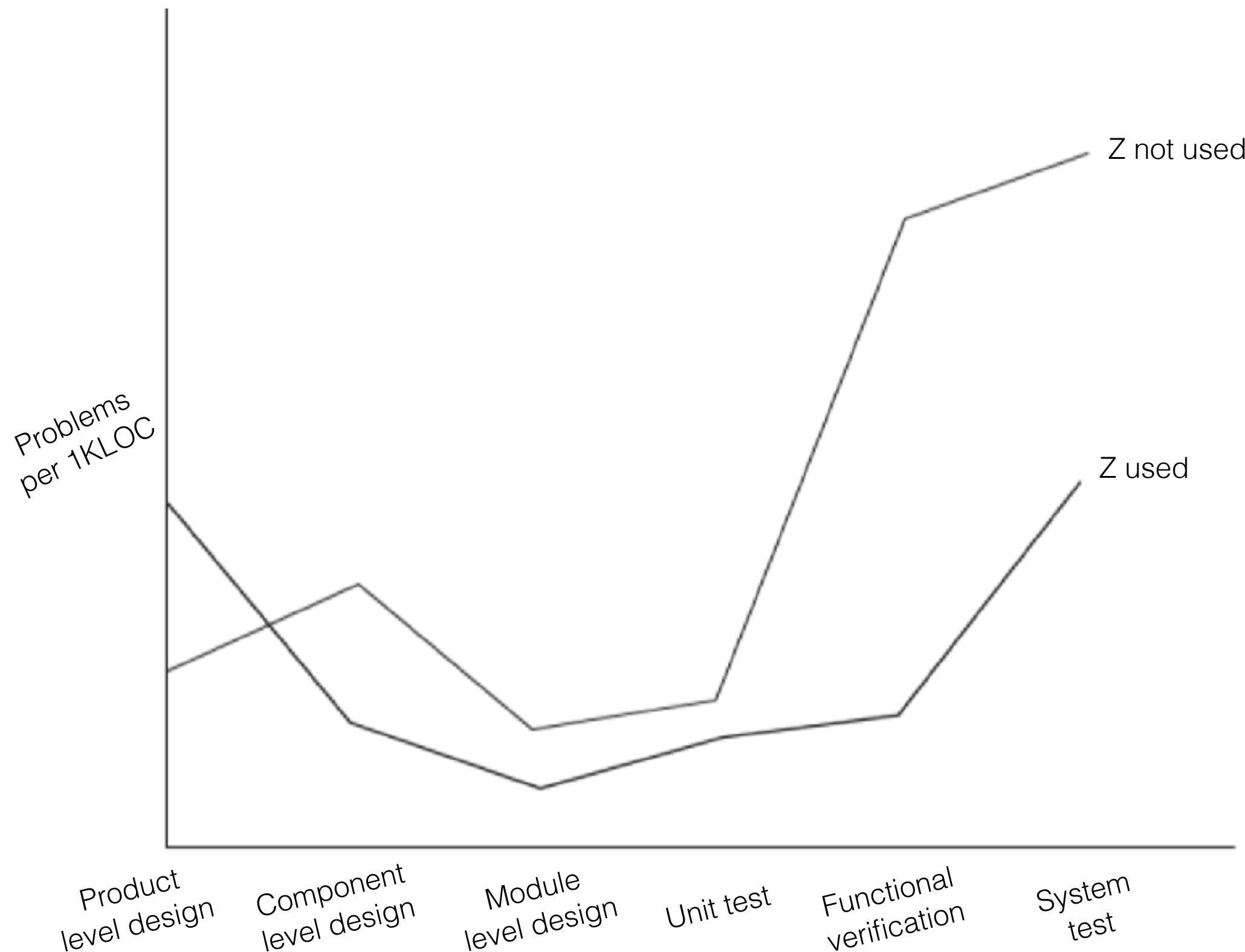
Abstract from Implementation

Machine-Checkable

Forces thinking up-front

Gets mistakes out of the way early
(i.e. when they are cheaper to fix)

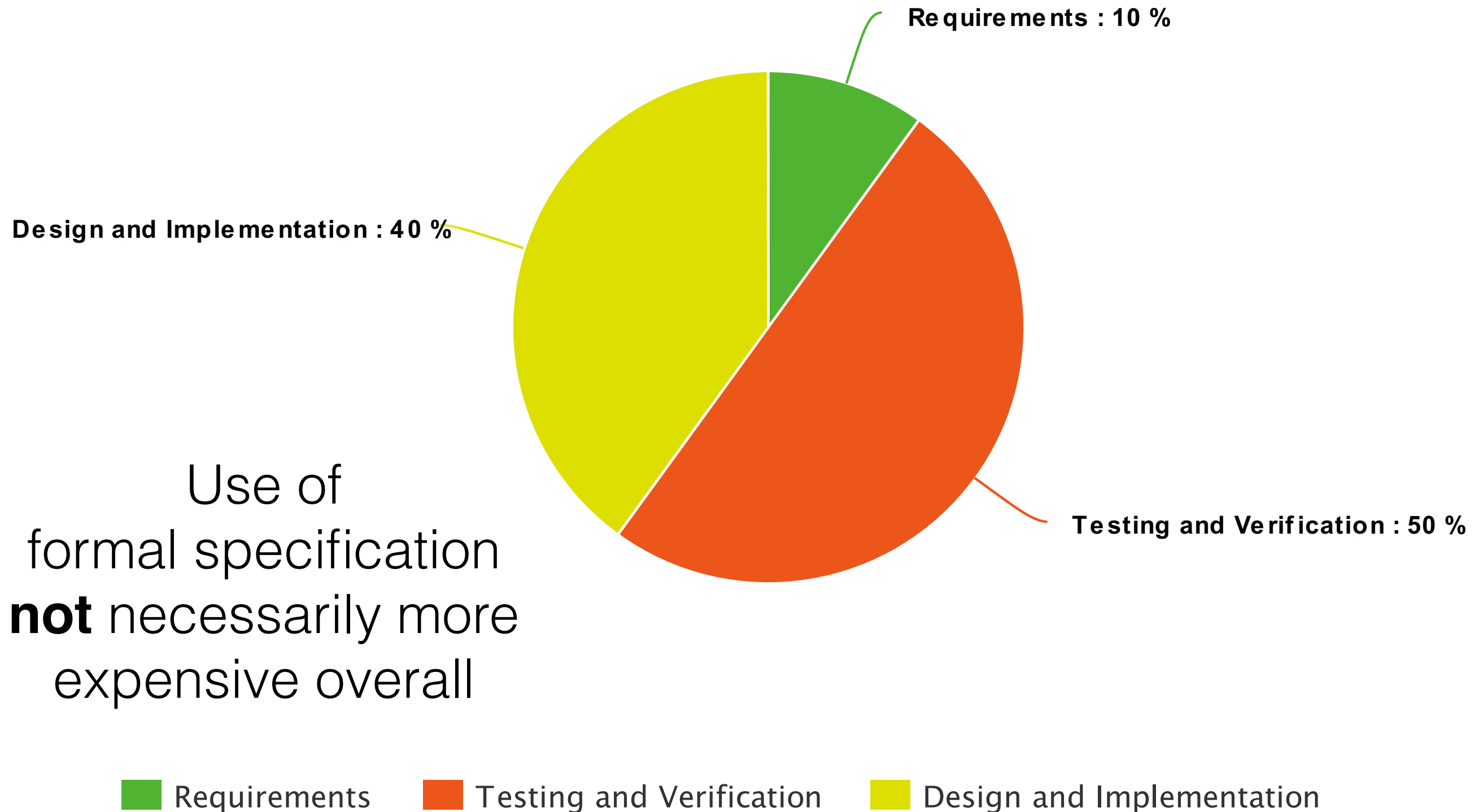
Effect on Cost



Effect on Cost



THE UNIVERSITY OF
MELBOURNE



meta-chart.com

Disadvantages

Requires specialised expertise

Might lengthen time to implementation

Loads more effort early in the development process

Not necessarily well suited to projects with
rapidly changing requirements.

(So it's important to get your HAZOP etc. correct)

Difficulty

All programmers know many formal languages

Specs are **shorter** and thus **simpler** to write than code

Specs say only **what** not **how**

Programs must say both

Ordinary programmers not trained
to write and read formal specs



SPECIFICATIONS IN ALLOY

Alloy



THE UNIVERSITY OF
MELBOURNE

Logic + Language + Tool

all $x : S \mid \dots$

```
module m1
open package/m2
...
pred p {
  ...
}
```

The screenshot displays the Alloy Analyzer interface. The main window shows a code editor with Alloy code for a password management system. The code includes assertions for deleting passwords and adding new ones, along with a `run` command to check the model. A solver status window in the top right corner reports the results of a SAT solver run, indicating that no counterexample was found. A visual model diagram in the foreground shows the state of the model, with nodes representing usernames, passwords, and URLs, and edges representing the relationships between them.

```
//If we add and then delete a password, we are back to the start
assert deleteIsUndo {
  all pb1, pb2, pb3: PassBook, url : URL, user : Username, p : Password {
    add [pb1, pb2, url, user, p] && delete [pb2, pb3, url, user, p]
    => pb1.password = pb3.password
  }
}

assert addAdds {
  all pb, pb': PassBook, url : URL, user : Username, p : Password {
    add [pb, pb', url, user, p] implies
      (no pb.password[user][url] or pb.password[user][url] == p)
  }
}

run add for 3 but 2 PassBook
run add for 5 URL, 5 Username, 10 Password, 2 PassBook
check updateWorks for 3 but 2 PassBook expect 0
check deleteIsUndo for 2 but 2 PassBook expect 0
check addAdds for 10 but 2 PassBook expect 0
```

Solver=sat4j Bitwidth=0 MaxSeq=0 SkolemDepth=1378 vars. 78 primary vars. 2764 clauses. 4ms.
No counterexample found. Assertion may be valid.

Executing "Check deleteIsUndo for 2 but 2 PassBook expect 0"
Solver=sat4j Bitwidth=0 MaxSeq=0 SkolemDepth=1378 vars. 78 primary vars. 2764 clauses. 4ms.
No counterexample found. Assertion may be valid.

Projected over PassBook

Visual Model Diagram:

- Nodes: Username0, Username1 (\$deleteIsUndo_user), Password0, Password1 (\$deleteIsUndo_pwd), URL0, URL1 (\$deleteIsUndo_url).
- Edges: password [url0] (Username0 to Password0), password [url0] (Username1 to Password0), password [url1] (Username1 to Password1).

Line 35, Column 7



THE UNIVERSITY OF
MELBOURNE

LASTPASS DEMO

Alloy Modelling Overview



signatures: declaring sets and relations

facts: axioms that hold over signatures and globally constrain the model to rule out nonsense

predicates: define relations between variables in a model, used to specify operations, state transitions

functions: shorthand for expressions